DB/ERD 설계서 v3.2

1. 목적

이 문서는 Harvest Slot 서비스의 실제 DB 테이블로 변환 가능한 ERD 설계서다. 최종 기준은 **19개 테이블**이며, 이 ERD는 개념 설명용이 아니라 MySQL DDL로 옮길 수 있는 논리 설계 기준이다.

이번 v3.2의 핵심은 다음이다.

```
어트리뷰트 사전 + 관계 카디널리티 문서만 보고
개발자가 Miro에서 ERD를 직접 재작성할 수 있어야 한다.
```

따라서 각 테이블은 엔티티 분류, PK, FK, UNIQUE, NULL 여부를 명시하고, 각 관계는 Peter Chen 방식의 카디널리티 표기와 FK 구현 방식을 함께 제공한다.

2. 최종 테이블 목록

번호	테이블	한글 엔티티명	분류	PK	설명
1	`accounts`	계정	카테고리 상위 엔티티 (Supertype Entity)	account_id	고객과 점주가 공통으로 사용하는 로그인 계정이다. role 값에 따라 고객 프로필 또는 점주 프로필 중 하나로 확장된다.
2	`customer_profiles`	고객 프로필	카테고리 하위 엔티티 (Subtype Entity)	customer_id	CUSTOMER 역할 계정의 고객 상세 정보다. 계정 1건당 최대 1건만 생성된다.
3	`owner_profiles`	점주 프로필	카테고리 하위 엔티티 (Subtype Entity)	owner_id	OWNER 역할 계정의 점주 상세 정보다. 계정 1건당 최대 1건만 생성된다.
4	`email_verifications`	이메일	엔티티(Entity)	email_verification_id	회원가입 또는 비밀번호 재설정 목적의 이메일 인증 코드 이력이다.

번호	테이블	한글 엔티티명	분류	PK	설명
		인증			
5	farms	농장	엔티티(Entity)	farm_id	점주가 운영하는 과수농가 정보다.
6	products	상품	엔티티(Entity)	product_id	농장에서 판매하는 과일 상품 정보다. 실제 예약 가능 수량은 harvest_slots에서 관리한다.
7	ml_predictions	ML 예측	엔티티(Entity)	prediction_id	수확 시기, 수확량, 예약 가능 범위, 추천 가격을 제안하는 ML 의사결정 보조 결과다.
8	harvest_slots	수확 슬롯	엔티티(Entity)	slot_id	점주가 고객에게 예약 가능하다고 확정한 수확 기간, 예약 가능 수량, 판매가다.
9	reservations	예약	엔티티(Entity)	reservation_id	고객이 수확 슬롯을 임시 점유한 예약 헤더다. 주문 전환 전까지 reserved_until으로 만료를 관리한다.
10	reservation_items	예약 품목	연관 엔티티(Associative Entity)	reservation_item_id	예약과 수확 슬롯 사이의 N:M 성격을 해소하는 예약 품목 상세다.
11	orders	주문	엔티티(Entity)	order_id	예약을 구매 의사로 확정한 주문 헤더다. 주문 하나는 반드시 예약 하나에서 전환된다.
12	order_items	주문 품목	연관 엔티티(Associative Entity)	order_item_id	주문과 예약 품목 전환 결과를 연결하는 주문 품목 상세다.

번호	테이블	한글 엔티티명	분류	PK	설명
13	`payments`	결제	엔티티(Entity)	payment_id	Mock 결제 요청과 승인 결과다. 결제 재시도를 고려해 주문 1건에 결제 기록 여러 건을 허용한다.
14	`procurements`	발주	엔티티(Entity)	procurement_id	결제 완료 후 점주에게 생성되는 출고/수확 처리 요청 헤더다.
15	`procurement_items`	발주 품목	연관 엔티티(Associative Entity)	procurement_item_id	발주와 주문 품목 전환 결과를 연결하는 발주 품목 상세다.
16	`quality_inspections`	신선도 검사	엔티티(Entity)	quality_inspection_id	발주 품목에 대해 점주 앱에서 촬영한 이미지의 DL 판별 결과와 점주 확정 판단이다.
17	`shipments`	배송	엔티티(Entity)	shipment_id	주문 단위 배송 정보다. MVP에서는 주문 1건당 배송 정보 최대 1건으로 처리한다.
18	`return_requests`	반품 요청	엔티티(Entity)	return_request_id	고객의 주문 단위 반품 요청과 점주 결정 결과다. MVP에서는 주문 1건당 반품 요청 최대 1건으로 처리한다.
19	`refunds`	환불	엔티티(Entity)	refund_id	승인된 반품 요청에 대한 Mock 환불 처리 결과다. 반품 요청 1건당 최대 1건, 결제 1건당 최대 1건으로 처리한다.

3. Miro ERD 작성 기준

3.1 엔티티 박스 작성 규칙

각 테이블은 하나의 엔티티 박스로 그린다. 박스 상단에는 다음 형식으로 적는다.

```
영문 테이블명 / 한글 엔티티명 / 분류
```

예시:

```
accounts / 계정 / 카테고리 상위 엔티티
```

박스 내부 어트리뷰트는 다음 형식으로 적는다.

```
[키/제약] 영문 컬럼명 : 개념 타입 / DB 타입 – 한글 어트리뷰트명
```

예시:

```
[PK] account_id : int / bigint – 계정 번호
[FK, UNIQUE, NOT NULL] account_id : int / bigint – 계정 번호
```

3.2 관계선 작성 규칙

관계선은 `assets/10_관계_카디널리티_전체.md`의 각 관계 카드에 있는 **Peter Chen 카디널리티 표기**를 따른다.

이 문서에서는 Miro/Mermaid의 Crow's Foot 기호를 ERD 선 라벨로 사용하지 않는다.

Miro에서 관계선을 그릴 때는 선 중앙 또는 선 근처에 다음 형식으로 적는다.

```
한글 관계명(english_relation_name) / 카디널리티
```

예시:

```
고객 프로필 확장(extends_customer_profile) / 1:1
예약 요청(requests_reservation) / 1:N
예약 ↔ 수확 슬롯 / N:M -> reservation_items로 해소
```

Peter Chen 표기	의미	선택성 확인 위치
`1:1`	1대1 관계	각 관계 카드의 `From 1개 기준 To 수`, `To 1개 기준 From 수`에서 `0..1` 또는 `1` 여부 확인
`1:N`	1대다 관계	각 관계 카드의 `0..N` 또는 `1..N` 여부 확인

Peter Chen 표기	의미	선택성 확인 위치
`N:M`	다대다 업무 관계	직접 DB 관계선으로 그리지 않고 연관 엔티티로 해소한다

중요: `1:1`, `1:N`은 관계의 큰 수량 형태를 보여주는 표기이고, `0..1`, `0..N`, `1..N`은 선택성과 필수성을 보여주는 상세 조건이다. 따라서 Miro 선에는 `1:1` 또는 `1:N`을 적고, 선택성은 관계 카드의 상세 항목을 기준으로 검토한다.

3.3 배타 프로필 규칙

`accounts`는 `customer_profiles` 또는 `owner_profiles`로 확장된다. 두 FK 관계는 DB에서 각각 UNIQUE로 구현하지만, 같은 계정이 두 프로필을 동시에 가지지 않도록 하는 배타 규칙은 서비스 로직에서 강제한다.

```
accounts.role = CUSTOMER -> customer_profiles 생성
accounts.role = OWNER    -> owner_profiles 생성
```

4. 원본 산출물

파일	역할
`assets/10_harvest_slot_final_19tables.dbml`	DBML 원본. 테이블, 컬럼, 제약, 관계 정의
`assets/10_harvest_slot_final_19tables.mmd`	Mermaid ERD 시각화
`assets/10_관계_카디널리티_전체.md`	Miro 재작성용 전체 관계와 카디널리티 문서
`assets/10_어트리뷰트_사전_19테이블.md`	Miro 재작성용 19개 테이블 전체 어트리뷰트 사전
`assets/10_schema_mysql.sql`	MySQL DDL 초안

5. 정규화 원칙

1. 주문 헤더와 주문 품목을 분리한다.
2. 예약 헤더와 예약 품목을 분리한다.
3. 발주 헤더와 발주 품목을 분리한다.
4. 고객/점주 프로필은 공통 계정에서 역할별 프로필로 분리한다.
5. 상품 정보는 수확 슬롯과 분리한다.
6. ML 예측 결과와 점주 확정 슬롯을 분리한다.
7. 수량·가격 스냅샷은 예약/주문 시점의 업무 사실로 보관한다.
8. 주문 단위 반품과 Mock 환불은 1:1 관계로 단순화한다.

6. ACID 적용 지점

예약 생성

```
BEGIN
SELECT harvest_slots WHERE slot_id IN (...) FOR UPDATE
검증: available_kg >= 요청 kg
```

```
INSERT reservations
INSERT reservation_items
UPDATE harvest_slots SET reserved_kg = reserved_kg + 요청 kg
COMMIT
```

주문 생성

```
BEGIN
SELECT reservations FOR UPDATE
검증: reservation_status = RESERVED, reserved_until > now
INSERT orders
INSERT order_items
UPDATE reservations SET reservation_status = ORDERED
COMMIT
```

Mock 결제 승인 및 발주 생성

```
BEGIN
SELECT orders FOR UPDATE
INSERT payments
UPDATE orders SET order_status = PAID, paid_at = now
UPDATE harvest_slots SET reserved_kg = reserved_kg - 주문 kg, sold_kg =
sold_kg + 주문 kg
INSERT procurements
INSERT procurement_items
UPDATE orders SET order_status = PROCUREMENT_REQUESTED
COMMIT
```

발주 결정

```
BEGIN
SELECT procurements FOR UPDATE
UPDATE procurement_items
UPDATE procurements
UPDATE orders
UPDATE order_items
COMMIT
```

반품 승인 및 환불

```
BEGIN
SELECT return_requests FOR UPDATE
UPDATE return_requests
INSERT refunds
```

```
UPDATE payments
UPDATE orders
COMMIT
```

7. 핵심 계산식

예약 가능 수량:

```
available_kg = confirmed_reservable_kg - reserved_kg - sold_kg
```

예약 품목 kg:

```
reserved_kg = package_count * products.package_unit_kg
```

예약 품목 금액:

```
subtotal_amount = package_count * harvest_slots.confirmed_price
```

8. Miro 재작성 순서

1. `accounts`, `customer_profiles`, `owner_profiles`, `email_verifications`를 먼저 배치한다.
2. `owner_profiles -> farms -> products -> harvest_slots` 순서로 농장/상품/슬롯 영역을 배치한다.
3. `ml_predictions`를 `farms`, `products`, `owner_profiles`, `harvest_slots` 사이에 배치한다.
4. `customer_profiles -> reservations -> reservation_items -> harvest_slots`를 배치한다.
5. `reservations -> orders -> order_items -> reservation_items`를 배치한다.
6. `orders -> payments`, `orders -> procurements -> procurement_items -> order_items`를 배치한다.
7. `procurement_items -> quality_inspections`, `orders -> shipments`, `orders -> return_requests -> refunds`, `payments -> refunds`를 배치한다.

9. 설계 기준

DB 기준은 `DBML → DDL → SQLAlchemy 모델 → Pydantic Schema → API → 화면` 순서로 맞춘다.